The OpenSpec + Superpowers workflow (AgenticApps standard)

The one document to read to understand how work happens in every AgenticApps repo. Copy it into each repo at `docs/WORKFLOW.md` and reference it from the host file. Companion docs: `GATE-INVENTORY.md` (every gate/hook mapped), `OPENSPEC-CLI-AND-MULTIHOST.md` (current CLI + agent-agnostic setup), `MEASUREMENT.md` (the quality-skill trial).

The one idea

Specs are the source of truth. Work happens as reviewed changes to the spec.

Three systems, three jobs, no overlap:

- **OpenSpec** owns *what the system is and what changes* — `specs/` (durable, current truth) and `changes/` (in-flight deltas). The spec/plan front end.
- **Superpowers** owns *how a change gets built* — brainstorm, writing-plans, TDD, code-review, verification-before-completion. The execution discipline.
- **Linear** owns *what's next and in what order* — roadmap, epics, priority. **Loose coupling: a change just references its Linear ID; there is no automated sync.**

GSD's `.planning/` engine is retired as the front end (kept only as read-only backup). Gitnexus is removed. Enforcement is one shell gate, wired as a per-agent hook **and** a CI/pre-commit backstop.

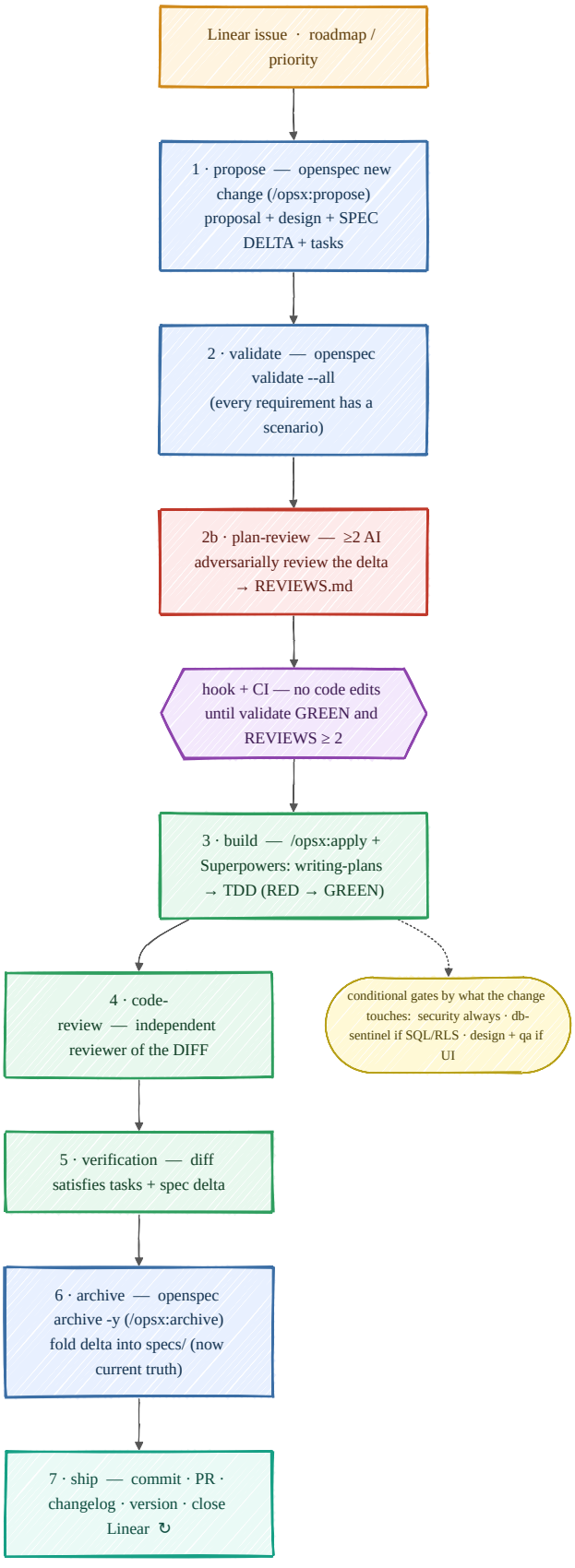
Tooling note (OpenSpec is CLI-driven now)

OpenSpec is a standalone CLI, run from a terminal — not something that lives "inside" an agent. Current shape (always confirm with `openspec --help`): `openspec init --tools claude,codex,opencode,pi`, `openspec new change <name>`, `openspec validate --all`, `openspec show`, `openspec archive <name> -y`, `openspec update`. Project context lives in `openspec/config.yaml` under `context:`. Adopt the **OPSX Core profile** (`/opsx:explore · propose · apply · archive`) — it maps 1:1 to the loop below; skip the Expanded profile. Details in `OPENSPEC-CLI-AND-MULTIHOST.md`.

The directory shape

```
<repo>/
├── openspec/
│   ├── config.yaml                 # product context (context:), profile, tools
│   ├── specs/<capability>/spec.md  # DURABLE current truth — what the system does now
│   └── changes/
│       ├── <active-change>/        # in-flight: proposal.md · design.md · specs/<cap>/spec.md (delta) · tasks.md
│       └── archive/<dated-change>/ # completed changes (history)
├── docs/
│   ├── WORKFLOW.md                 # this file
│   ├── decisions/                  # ADRs — your convention, UNCHANGED (see "ADRs" below)
│   └── legacy-planning/            # the old .planning/, kept read-only as backup
├── AGENTS.md  or  CLAUDE.md        # PROCESS only (discipline, how to work) — no product spec
├── .<host>/…                       # per-agent hook → calls the global gate script
└── (CI + git pre-commit)           # same gate script — the agent-agnostic enforcement floor
```

`specs/` answers "what does this do?" in one file per capability. `changes/archive/` answers "how did it get that way?". `docs/decisions/` answers "why?". Linear answers "what's next?".



The loop, at a glance — one change from Linear issue to ship. Colours map to the layer legend below.

Linear · roadmap | OpenSpec · spec & lifecycle | Superpowers · execution | adversarial review | enforcement hook | ship

The loop (every code-touching task)

```
  Linear issue (roadmap)
       │  change references its Linear ID — no sync
       ▼
1 · propose      →  openspec new change <name>  (/opsx:propose)
    │               writes proposal.md + design.md + SPEC DELTA + tasks.md
    ▼
2 · validate     →  openspec validate --all   (STRUCTURAL: every requirement has a scenario)
    ▼
2b · plan-review →  ≥2 other-vendor agents adversarially review the change
    │               (proposal + spec delta) → changes/<name>/REVIEWS.md   [keeps ADR-0018]
    ▼  ══ gate: no code edit until validate GREEN and REVIEWS.md ≥2 — hook + CI backstop ══
3 · build        →  /opsx:apply + Superpowers: brainstorm? → writing-plans → TDD (RED→GREEN)
    │               conditional gates: security always · db-sentinel if SQL/RLS · design/qa if UI
    ▼
4 · code-review  →  Superpowers independent reviewer of the DIFF (Stage 2)
    ▼
5 · verification →  verification-before-completion: diff satisfies tasks + spec delta
    ▼
6 · archive      →  openspec archive <name> -y  (/opsx:archive) — folds delta into specs/
    ▼
7 · ship         →  commit · PR · changelog · version · close Linear   (OpenSpec does NOT do this)
       ↻ next change
```

There are **two independent reviews**, and both are kept: **plan-review** (2b, multi-AI) critiques the *plan/spec delta* before code; **code-review** (4, Superpowers Stage 2) critiques the *implementation diff* before archive. `openspec validate` is a structural lint, not a review — it does not replace either.

Step by step

1. **Propose.** State the change as a delta to the spec *before* writing code. `proposal.md` = Why / What Changes / Impact (+ `Linear: AGE-123`); `design.md` = the technical approach; the delta in `changes/<name>/specs/<cap>/spec.md` names the ADDED/MODIFIED/REMOVED requirements; `tasks.md` is the checklist.
2. **Validate (structural).** `openspec validate --all` fails if any requirement lacks a scenario or the delta is malformed. A lint, not a review. 2b. **Plan-review (adversarial, kept).** ≥2 other-vendor agents critique the change and write `changes/<name>/REVIEWS.md`. Your existing multi-AI gate, retargeted from `PLAN.md` to the change. Dropping it re-opens the ADR-0018 failure mode. Escape hatches (`GSD_SKIP_REVIEWS=1` , a per-change skip marker) carry over.
3. **Build with Superpowers.** Only brainstorm when the change is genuinely open-ended. Always: writing-plans → failing test first (RED) → make it pass (GREEN). Conditional gates fire by what the diff touches (security always; db-sentinel on SQL/RLS; design/qa on UI).
4. **Code-review (adversarial, kept).** Superpowers Stage-2 independent reviewer critiques the diff against the spec delta. This is the "check the implementation" review — it is *not* covered by OpenSpec.
5. **Verify.** `verification-before-completion` — the diff actually satisfies the tasks and the spec delta.
6. **Archive.** `openspec archive` folds the delta into `specs/` so the spec is current and moves the change to `archive/` .
7. **Ship (kept, thin).** Conventional commit · PR · changelog (from `proposal.md`) · version bump · close the Linear issue. **`archive ≠ ship`** — OpenSpec does not commit, PR, tag, or version.

Task-size routing (kept): tiny/small changes move fast and may skip the heavy gates; **medium/large changes require plan-review + code-review + an ADR** for any locked design decision. Same rule as today, now keyed to the change instead of the phase.

The gates, simplified

Old gate	Now
spec-review (structural)	folds into `openspec validate`
plan-review (multi-AI adversarial)	**KEPT** — retargeted to review the change (proposal + spec delta); gate requires `validate` green AND `REVIEWS.md` ≥2. Not covered by OpenSpec.
cso (security)	keep — always-on for product repos
database-sentinel	keep — conditional (change touches SQL/RLS)
qa / design-critique / design-shotgun / impeccable	keep — conditional (change touches UI); impeccable + Go skills on a measured trial (`MEASUREMENT.md`)
ts-declare-first	demoted to a CI lint, not a gate

Kept as-is from Superpowers (not shown above): code-review (independent diff reviewer), tdd, verification-before-completion, brainstorming, branch-close. Always-on = **validate + plan-review + security + tests**; everything else is conditional on what the diff touches.

Enforcement (hook + CI backstop)

The gate is **one host-agnostic shell script** — `validate` must be green **and** the active change must carry `REVIEWS.md` with ≥2 reviewers. Wire it two ways:

- **Per-agent `PreToolUse` hook** (claude `settings.json` · codex `.codex/hooks.json` · opencode · pi) → fast feedback that blocks an edit in-session. A PreToolUse hook cannot gate *its own installing session* (proven in the pilot), so it is convenience, not the guarantee.
- **git pre-commit + CI check** running the same script → the **agent-agnostic enforcement floor**: it catches an edit from *any* agent or a human, regardless of hooks. This is the reliable layer; ship it first.

What goes where (the rule that keeps it clean)

- A sentence that says **what the product does or must guarantee** → `specs/`. ("The LLM never produces a numeric score." "Access to an unowned resource returns 404.")
- A sentence that says **how you should work** → AGENTS.md / CLAUDE.md (or the workflow skill). ("Write the failing test first." "Touch only what the task names.")
- **Why a locked architectural decision was made** → an ADR in `docs/decisions/`.
- Effort history (old phase notes, research logs) → `docs/legacy-planning/`, read-only.

Keep product truth out of the host file and process out of the spec, and both stay small.

ADRs in the new workflow

ADRs are **your own convention** (industry-standard "Architecture Decision Records"), not a GSD artifact — GSD logged decisions inside `.planning/`, but the durable `docs/decisions/NNNN-slug.md` files are the shared, host-neutral AgenticApps standard. They are **fully respected and unchanged**, and OpenSpec is orthogonal to them:

- `specs/` = *what* the system does · `changes/archive/` = *what changed & when* · `docs/decisions/` (ADRs) = *why* a significant/expensive-to-reverse decision was made · a change's `proposal.md` / `design.md` = the *per-change* rationale (archived with it).
- **Specs cite ADRs** inline (e.g. "the LLM never produces a numeric score *(ADR-0008)*").
- **Keep creating them:** routine rationale lives in the proposal; when a change **locks a durable architectural decision**, promote that rationale into a numbered ADR and have the spec reference it. Medium/large changes still require one (task-size routing).
- The migration itself is authored *as* ADRs (the standard's adoption ADR supersedes the GSD-binding ADRs). Nothing is lost; the ADR↔spec link becomes first-class.

Multi-host & agent-agnostic

The workflow is **agent-agnostic by construction**, because its core is three neutral primitives: the **CLI** (a standalone binary every agent and human calls), the **spec files** (`openspec/` — plain markdown any agent reads), and the **gate shell script**. The CLI runs *outside* the agent, so no agent is privileged.

- **All four hosts are first-class.** OpenSpec supports 28 tools including claude, codex, opencode, pi — generate every present host's command files in one shot: `openspec update --tools claude,codex,opencode,pi` . Bound upstream (linked), not re-ported; `specs/` are host-neutral, so a claude session and a codex session on the same repo read identical truth.
- **Global (install once, agnostic):** the `openspec` CLI · the gate shell script · the multi-AI reviewer wrapper · the discipline prose.
- **Repo-specific:** `openspec/` (specs, changes, `config.yaml`) · the generated per-tool commands · the hook + CI wiring.
- **An unsupported agent still works** via the CLI + `AGENTS.md` + the spec files — it just loses the `/opsx:*` slash sugar. *(Open item: opencode + pi hook surfaces are unconfirmed — verify during their host rollout.)*

Also kept (cross-cutting — see GATE-INVENTORY.md)

Orthogonal to OpenSpec and **all retained**: the commitment ritual + 4 coding-discipline rules · knowledge-capture to Obsidian (retargeted "phase completion" → "change archive") · the observability generator · the prompt-injection guard · the gitleaks secret scan · session-handoff · task-size routing · the migration framework. Nothing in the migration removes these — `GATE-INVENTORY.md` gives each a verdict and its new home.

Why this is better (short version)

The agent reads one current, validated spec instead of reconciling a pile of phase notes — precise input is the whole game (the pilot's multi-AI review even caught a real spec defect *before code*). Drift is caught by `validate` and forced shut by `archive` , instead of surfacing in a milestone audit months later. Real work is a delta, and "change" is the native unit — no more fractional `03.5 / 04.7 INSERTED` phase numbers. And the spec is the portable, agent-agnostic artifact across all four hosts.